

運用 Firestore、Vector Search 和 Gemini 2.0，打造可依情境推薦瑜珈體式的應用程式 (Java 版本)！

來源：<https://codelabs.developers.google.com/codelabs/smart-yoga-pose-search-firestore?hl=zh-tw>

步驟數：10

在本程式碼研究室中，您將建構知識導向的瑜珈姿勢搜尋應用程式，旨在回答使用者關於瑜珈姿勢的問題，並與使用者的問題相符。也能執行建立和編輯瑜珈姿勢等管理工作。

目錄

1. [總覽](#)
2. [事前準備](#)
3. [資料庫設定](#)
4. [Firestore REST API](#)
5. [資料擷取](#)
6. [Vector Search](#)
7. [Gemini 2.0 Flash](#)
8. [執行應用程式](#)
9. [清除所用資源](#)
10. [恭喜](#)

1. 總覽

在健康與健身應用程式領域，為使用者提供豐富且引人入勝的體驗至關重要。以瑜珈應用程式為例，這表示除了提供簡單的體位文字說明，還能提供全面資訊、多媒體內容和智慧搜尋功能。在本網誌中，我們將探討如何使用 Google Cloud 的 Firestore 建構強大的瑜珈體式資料庫、運用其向量搜尋擴充功能進行情境比對，以及整合 Gemini 2.0 Flash (實驗版) 的強大功能來處理多模態內容。

為什麼要使用 Firestore ？

Firestore 是 Google Cloud 的無伺服器 NoSQL 文件資料庫，非常適合建構可擴充的動態應用程式。以下說明為何這項功能非常適合我們的 Yoga 應用程式：

- **擴充性和效能：**Firestore 會自動調整資源配置，處理數百萬名使用者和大型資料集，確保應用程式在成長時仍能保持回應速度。
- **即時更新：**內建的即時同步功能可確保所有連結的用戶端資料一致，非常適合用於即時課程或協作練習等功能。
- **彈性資料模型：**Firestore 的文件式結構可讓您儲存各種資料類型，包括文字、圖片，甚至是嵌入內容，非常適合用來呈現複雜的瑜珈姿勢資訊。
- **強大的查詢功能：**Firestore 支援複雜查詢，包括等式、不等式，以及現在透過新擴充功能支援的[向量相似度搜尋](#)。
- **離線支援：**Firestore 會在本機快取資料，因此即使使用者離線，應用程式仍可正常運作。

使用 Firestore 向量搜尋擴充功能提升搜尋體驗

處理瑜珈體位等複雜概念時，傳統的關鍵字搜尋可能會受到限制。使用者可能想搜尋「打開髖關節」或「改善平衡感」的體式，但不知道具體名稱。這時向量搜尋就派上用場了。

透過 Firestore 進行向量搜尋，您可以：

- **生成嵌入：**使用 Vertex AI 或自訂模型，將文字說明 (未來可能包括圖片和音訊) 轉換為數值向量表示法 (嵌入)，藉此提取語意。
- **儲存嵌入：**直接在 Firestore 文件中儲存這些嵌入。
- **執行相似度搜尋：**查詢資料庫，找出與指定查詢向量語意相似的文件，進而進行情境比對。

整合 Gemini 2.0 Flash (實驗版)

[Gemini 2.0 Flash](#) 是 Google 最先進的多模態 AI 模型，這項技術目前仍處於實驗階段，但可望為 Yoga 應用程式帶來豐富的體驗：

- **生成文字：**使用 Gemini 2.0 Flash 生成瑜珈體位的詳細說明，包括益處、修改方式和禁忌症。
- **圖像生成 (模擬)：**雖然 Gemini 尚未開放直接圖像生成，但我已使用 Google 的 Imagen 模擬這項功能，生成可視為姿勢代表的圖像。
- **生成音訊 (模仿)：**同樣地，我們可以使用文字轉語音 (TTS) 服務，為每個姿勢建立語音指示，引導使用者完成練習。

我可能會提議整合應用程式，以使用模型的下列功能：

- **多模態 Live API：**這項新 API 可協助您建立即時影像和音訊串流應用程式，並使用工具。
- **速度和效能：**相較於 Gemini 1.5 Flash，Gemini 2.0 Flash 的首個權杖輸出時間 (TTFT) 大幅縮短。
- **提升代理程式體驗：**Gemini 2.0 在多模態理解、程式設計、遵循複雜指令和函式呼叫方面都有所提升。這些改善項目共同支援更出色的代理體驗。

詳情請參閱這份[說明文件](#)頁面。

以 Google 搜尋建立基準

為提升可信度及提供更多資源，我們可以整合 Google 搜尋，為應用程式提供的資訊建立基準。這表示：

- **情境搜尋**：管理員使用者輸入姿勢詳細資料時，我們可以透過姿勢名稱執行 Google 搜尋。
- **網址擷取**：從搜尋結果中，我們可以擷取相關網址 (例如文章、影片或信譽良好的瑜珈網站)，並在應用程式中顯示。

建構項目

本實驗室的學習內容包括：

1. 建立 Firestore 集合並載入 Yoga 文件
2. 瞭解如何使用 Firestore 建立 CRUD 應用程式
3. 使用 Gemini 2.0 Flash 生成瑜珈體式說明
4. 啟用 Firebase Vector Search 與 Firestore 的整合功能
5. 根據瑜珈說明生成嵌入項目
6. 對使用者搜尋文字執行相似度搜尋

需求條件

- [Chrome](#) 或 [Firefox](#) 瀏覽器
 - 已啟用計費功能的 Google Cloud 專案。
-

2. 事前準備

建立專案

1. 在 [Google Cloud 控制台](#) 的專案選取器頁面中，選取或建立 Google Cloud [專案](#)。
2. 確認 Cloud 專案已啟用計費功能。瞭解如何[檢查專案是否已啟用計費功能](#)。
3. 您將使用 [Cloud Shell](#)，這是 Google Cloud 中執行的指令列環境，預先載入了 bq。點選 Google Cloud 控制台頂端的「啟用 Cloud Shell」。



4. 連至 Cloud Shell 後，請使用下列指令確認驗證已完成，專案也已設為獲派的專案 ID：

```
gcloud auth list
```

5. 在 Cloud Shell 中執行下列指令，確認 gcloud 指令已瞭解您的專案。

```
gcloud config list project
```

6. 如果未設定專案，請使用下列指令來設定：

```
gcloud config set project <YOUR_PROJECT_ID>
```

7. 按照這個[連結](#)啟用必要的 API，直到可以點選「啟用」按鈕為止。

如果遺漏任何 API，您隨時可以在導入過程中啟用。

如要瞭解 gcloud 指令和用法，請參閱[說明文件](#)。

3. 資料庫設定

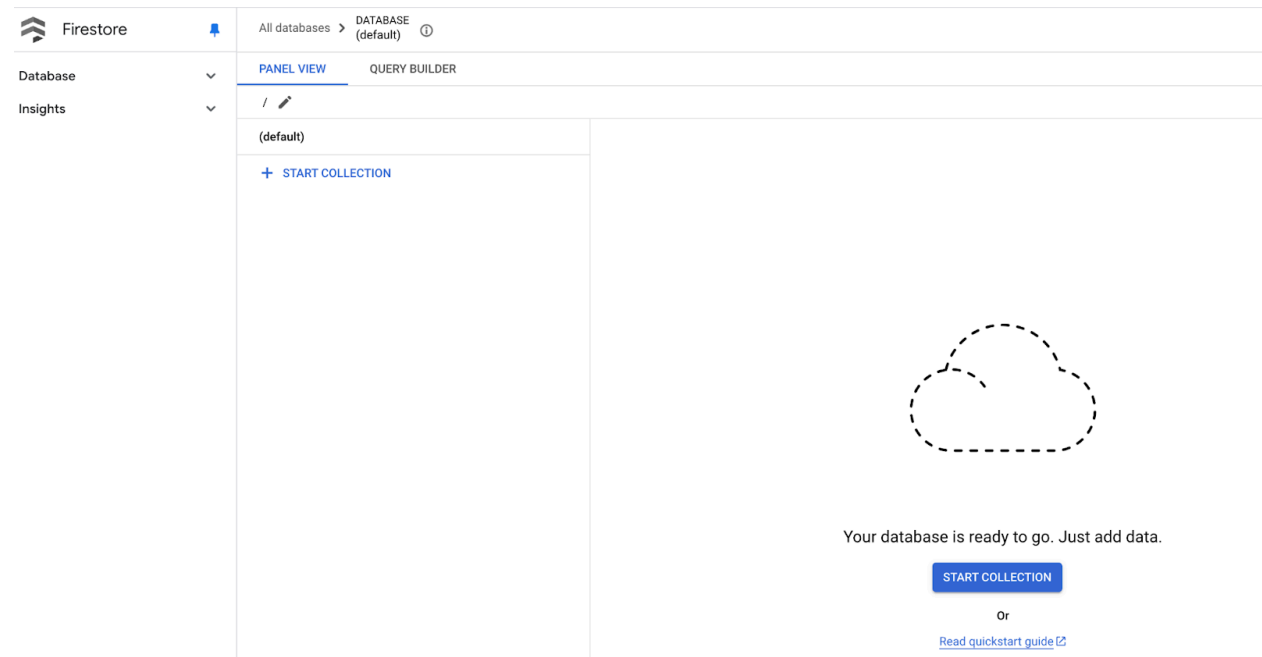
如要瞭解如何設定 Firestore 執行個體，請參閱說明文件中的[步驟](#)。大致來說，我會先按照下列步驟操作：

前往 [Firestore 檢視器](#)，然後在「選取資料庫服務」畫面中，選擇原生模式的 Firestore

- 2. 選取 Firestore 的位置 (請務必選擇 **us-central1**，並在整個程式碼研究室中選擇區域 / 位置時遵循此做法)
- 3. 按一下「建立資料庫」(如果是第一次，請保留預設的「(default)」資料庫)

建立 Firestore 專案時，系統也會在 Cloud API 管理工具中啟用 API

- 4. 重要事項：請選擇安全規則的「測試」版本 (而非「正式」版本)，確保資料可供存取
- 5. 設定完成後，您應該會看到 Native 模式的 Firestore 資料庫、集合和文件檢視畫面，如下圖所示：



- 6. 請先不要執行這個步驟，但為了記錄，您可以點選「開始收藏」並建立新集合。將集合 ID 設為「poses」。按一下「儲存」按鈕。

Start a collection

A collection is a set of one or more documents that contain data. Start a collection at this path by adding its first document. [Learn more](#)

Give the collection an ID

Parent path
/

Collection ID *

Choose an ID that describes the documents that you'll add to this collection.

Add its first document ?

Document ID

Assigned automatically. Customise if needed.

Field name

Field type

Field value

string

+ ADD FIELD

SAVE

SAVE & ADD ANOTHER

CANCEL

製作應用程式的專業訣竅：

1. 確定資料模型並找出應有權存取不同類型文件的使用者後，您就可以透過 Firebase 介面建立、編輯及監控[安全性規則](#)。您可以透過這個連結存取安全性規則：
https://console.firebase.google.com/u/0/project/<<your_project_id>>/firestore/rules
2. 請務必在從開發階段部署 / 推出專案前，編輯、監控及[測試](#)安全規則，因為這通常是導致應用程式運作方式不同的幕後原因：)

這次示範會使用測試模式。

4. Firestore REST API

1. REST API 適用於下列用途：a. 從資源受限的環境存取 Firestore，無法執行完整的用戶端程式庫。自動執行資料庫管理作業，或擷取詳細的資料庫中繼資料
2. 使用原生用戶端程式庫是使用 Firestore 最簡單的方式，但在某些情況下，直接呼叫 REST API 會很有用
3. 在本網誌中，您會看到 Firestore REST API 的用法和示範，而非原生用戶端程式庫
4. 如要進行驗證，Firestore REST API 接受 Firebase 驗證 ID 權杖或 Google Identity OAuth 2.0 權杖。如要進一步瞭解驗證和授權主題，請參閱[說明文件](#)。
5. 所有 REST API 端點都位於基本網址 <https://firestore.googleapis.com/v1/> 下。

Spring Boot 和 Firestore API

這個 Spring Boot 框架解決方案會示範用戶端應用程式如何使用 Firestore API，以互動式使用者體驗收集及修改瑜珈姿勢和呼吸細節。

如要詳細瞭解 Yoga 姿勢應用程式的 Firestore CRUD 解決方案部分，請參閱[這篇網誌](#)。

如要專注於目前的解決方案，並隨時隨地瞭解 CRUD 部分，請從 Cloud Shell 終端機複製下列存放區中專注於這篇網誌的完整解決方案，並取得程式碼集副本。

```
git clone https://github.com/AbiramiSukumaran/firestore-poserecommender
```

注意事項：

1. 複製這個存放區後，您只需要針對專案 ID、API 等進行幾項變更，不需要進行其他變更，就能啟動及執行應用程式。我們會在後續章節中說明應用程式的各個元件。變更項目如下：
2. 在 `src/main/java/com/example/demo/GenerateImageSample.java` 檔案中，將「<<YOUR_PROJECT_ID>>」替換為您的專案 ID
3. 在 `src/main/java/com/example/demo/GenerateEmbeddings.java` 檔案中，將「<<YOUR_PROJECT_ID>>」替換為您的專案 ID
4. 在 `src/main/java/com/example/demo/PoseController.java` 中，將所有「<<YOUR_PROJECT_ID>>」和資料庫名稱 (本例中為 `"(default)"`) 替換為設定中的適當值：,)
5. 在 `src/main/java/com/example/demo/PoseController.java` 中，將「[YOUR_API_KEY]」替換為 Gemini 2.0 Flash 的 API 金鑰。您可以從 [AI Studio](#) 取得這項資訊。
6. 如要在本機測試，請在 Cloud Shell 終端機中，從專案資料夾執行下列指令：

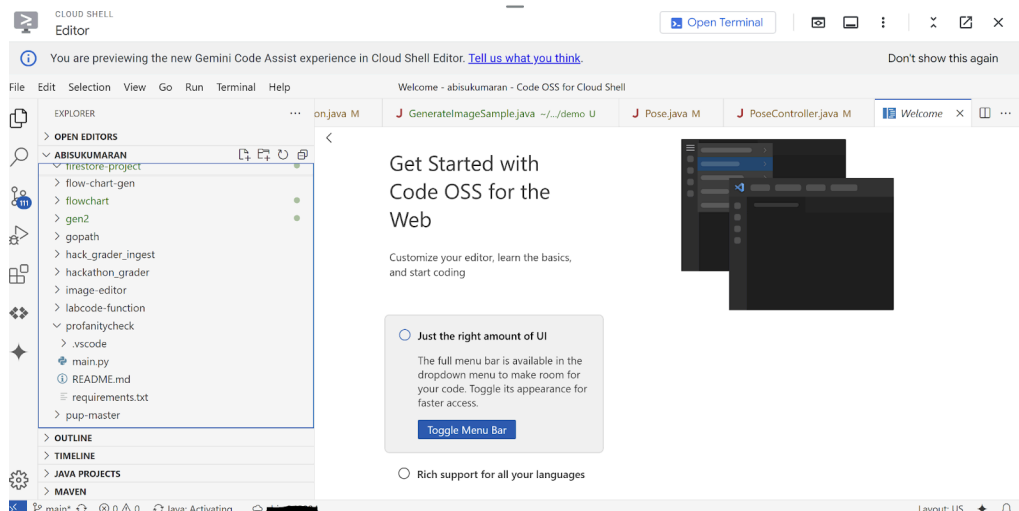
```
mvn package

mvn spring-boot:run
```

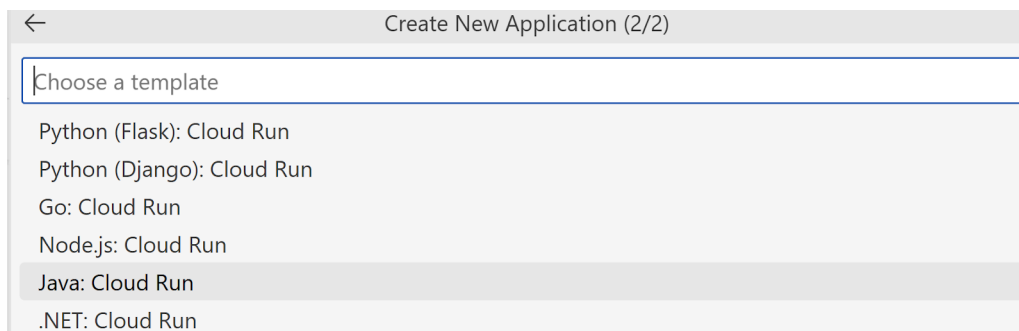
目前，您可以點選 Cloud Shell 終端機中的「網頁預覽」選項，查看應用程式的執行情況。我們尚未準備好執行測試及試用應用程式。

3. **選用步驟：**如要在 Cloud Run 中部署應用程式，您必須從 Cloud Shell 編輯器從頭啟動全新的 Java Cloud Run 應用程式，並將 [存放區](#)中的 `src` 檔案和範本檔案新增至新專案的相應資料夾 (因為目前的 GitHub 存放區專案預設未設定 Cloud Run 部署設定)。在這種情況下，請按照下列步驟操作 (而非複製現有存放區)：

4. 前往 **Cloud Shell 編輯器** (請確認開啟的是編輯器，而非終端機)，然後點選狀態列左側的 Google Cloud 專案名稱圖示 (下圖中已遮蓋的部分)



2. 從選項清單中依序選取「New application」->「Cloud Run Application」->「Java: Cloud Run」，並將其命名為「firestore-poserecommender」



3. 現在您應該會看到 Java Cloud Run 應用程式的完整堆疊範本，且已預先設定完畢，隨時可供使用
4. 移除現有的 Controller 類別，然後將下列檔案複製到專案結構中的對應資料夾：

firestore-poserecommender/src/main/java/com/example/demo/

1. FirestoreSampleApplication.java
2. GenerateEmbeddings.java
3. GenerateImageSample.java
4. Pose.java
5. PoseController.java
6. ServletInitializer.java

firestore-poserecommender/src/main/resources/static/

7. Index.html

firestore-poserecommender/src/main/resources/templates/

8. contextsearch.html
9. createpose.html
10. errmessage.html

11. pose.html

12. ryoq.html

13. searchpose.html

14. showmessage.html

firestore-poserecommender/

15. Dockerfile

16. 您需要在對應檔案中進行變更，將專案 ID 和 API 金鑰替換為各自的值。(上述步驟 1a、1b、1c 和 1d)。

5. 資料擷取

應用程式的資料位於這個檔案 data.json：<https://github.com/AbiramiSukumaran/firestore-poserecommender/blob/main/data.json>

如要從一些預先定義的資料開始，可以複製 JSON，並將所有「<<YOUR_PROJECT_ID>>」的出現位置替換為您的值

- 前往 [Firestore Studio](#)
- 請確認您已建立名為「poses」的集合
- 從上述存放區檔案逐一新增文件

或者，您也可以按照下列步驟，從我們為您建立的預先定義集合中一次匯入資料：

1. 前往 Cloud Shell 終端機，確認已設定有效的 Google Cloud 專案，且您已獲得授權。使用下列 **gsutil** 指令，在專案中建立 bucket。在下列指令中，將 **<PROJECT_ID>** 變數替換為您的 Google Cloud 專案 ID：

```
gsutil mb -l us gs://<PROJECT_ID>-yoga-poses-bucket
```

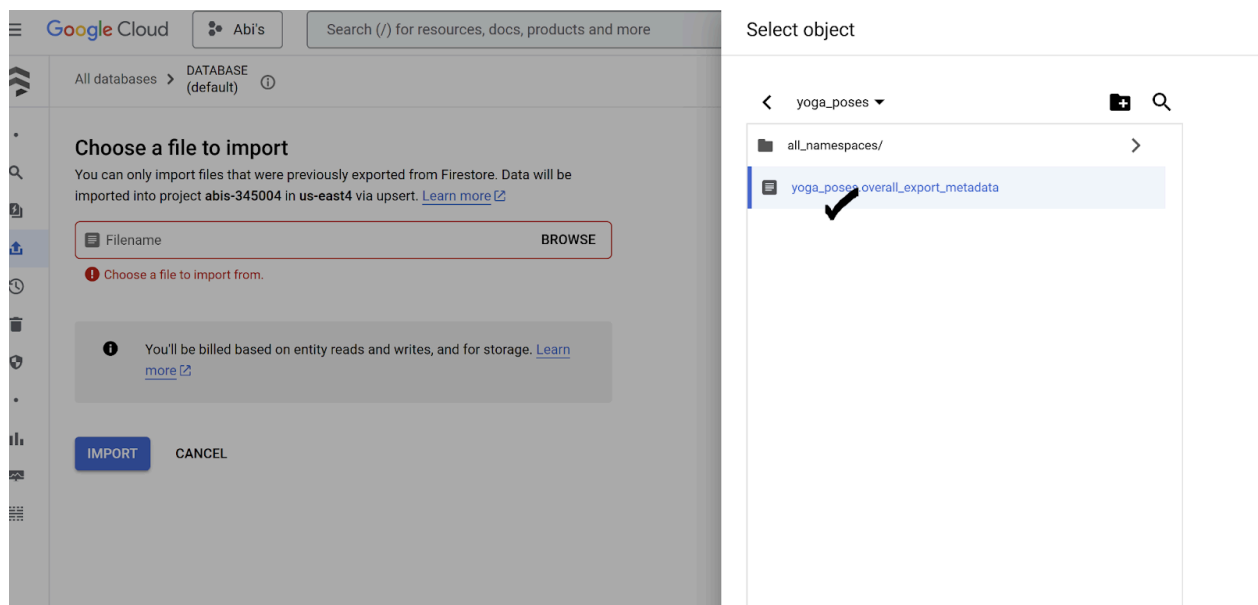
2. 建立好 bucket 後，我們需要將準備好的資料庫匯出內容複製到這個 bucket，才能匯入 Firebase 資料庫。使用下列指令：

```
gsutil cp -r gs://demo-bq-gemini-public/yoga_poses gs://<PROJECT_ID>-yoga-poses-bucket
```

現在我們已準備好要匯入的資料，可以進入最後一個步驟，將資料匯入我們建立的 Firebase 資料庫 (**default**)。

3. 前往 Firestore 控制台，然後按一下左側導覽選單中的「匯入/匯出」。

選取「匯入」，然後選擇您剛建立的 Cloud Storage 路徑，並導覽至可選取「yoga_poses.overall_export_metadata」檔案的位置：



4. 按一下 [匯入]。

匯入作業需要幾秒鐘，完成後請前往 <https://console.cloud.google.com/firestore/databases>，選取「default」資料庫和「poses」集合，驗證 Firestore 資料庫和集合，如下所示：

5. 您也可以在透過「建立新姿勢」動作部署後，透過應用程式手動建立記錄。

步驟 6 · 約 0 分鐘

6. Vector Search

啟用 Firestore 向量搜尋擴充功能

使用這項**擴充功能**，即可透過全新的向量搜尋功能，自動嵌入及查詢 Firestore 文件！系統會將你導向 Firebase Extensions Hub。

安裝向量搜尋擴充功能時，請指定集合和文件欄位名稱。新增或更新含有這個欄位的文件時，這項擴充功能會計算文件的向量嵌入。這個向量嵌入會寫回相同的文件，且該文件會在向量存放區中建立索引，以供查詢。

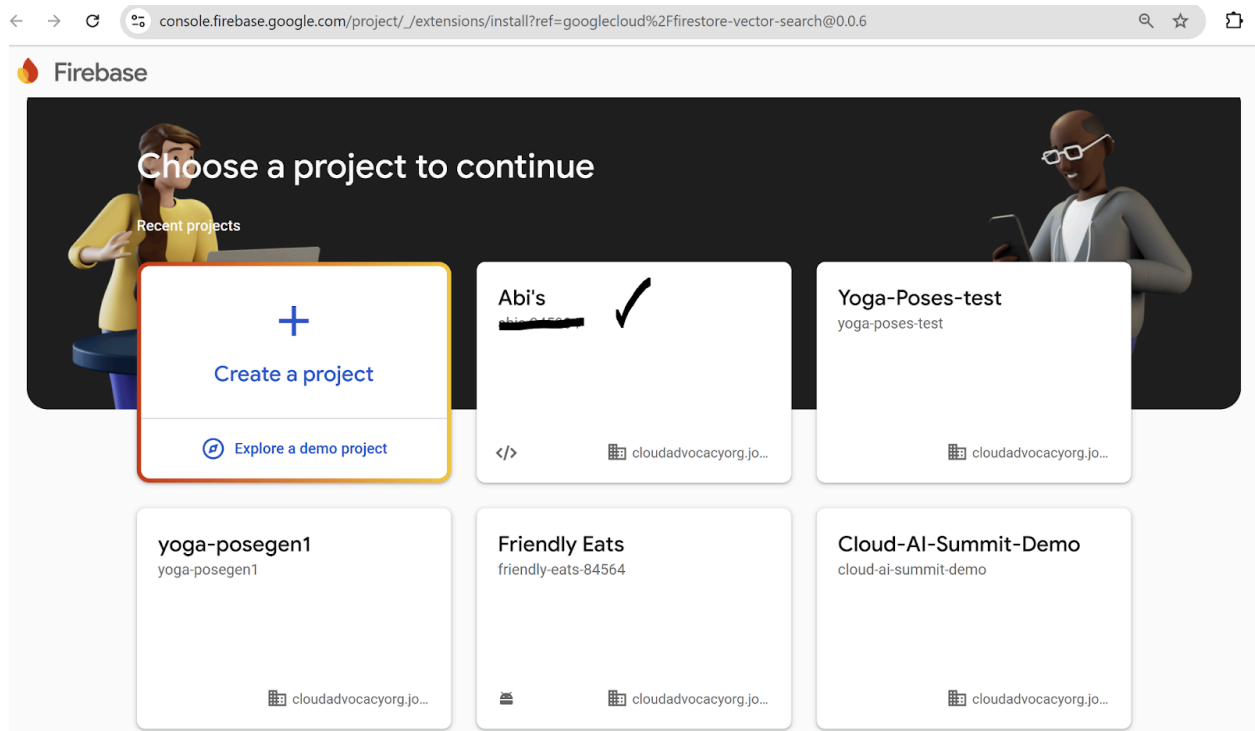
請按照下列步驟操作：

安裝擴充功能：

從 Firebase Extensions Marketplace 安裝「Vector Search with Firestore」擴充功能，方法是按一下「Install in Firebase Console」。

重要事項：

首次前往這個擴充功能頁面時，您需要選取與 Firebase 控制台所列 Google Cloud 控制台相同的專案。



如果專案未列出，請在 Firebase 中新增專案 (從清單中選擇現有的 Google Cloud 專案)。

設定擴充功能：

指定集合 (「poses」)、包含要嵌入文字的欄位 (「posture」)，以及嵌入維度等其他參數。

如果這個步驟列出需要啟用的 API，設定頁面會允許您啟用，請按照相應步驟操作。

啟用 API 後，如果頁面過一段時間仍未回應，請重新整理頁面，即可看到已啟用的 API。

× Install this extension

From googlecloud, in project 'Abi's'



Vector Search with Firestore

googlecloud/firestore-vector-search@0.0.6

Perform vector similarity search with Firestore

[Extension details](#) [View source](#)



Update extension instance ID

Extension instance ID updated to firestore-vector-search-ywdd



Review billing and usage

This extension uses billable Firebase services



Review APIs enabled and resources created

Uses Vertex AI API, Cloud Tasks API, and Eventarc API. Creates 7 Cloud Functions.



Review access granted to this extension

Cloud Datastore User, Vertex AI User, Storage Object Admin, Cloud Datastore Index Admin, Secret Manager

在其中一個後續步驟中，您可以選擇使用 LLM 生成嵌入內容。選擇「Vertex AI」。

× Install this extension

From googlecloud, in project 'Abi's'

Uses Vertex AI API, Cloud Tasks API and Eventarc API. Creates 7 Cloud Functions.



Review access granted to this extension

Cloud Datastore User, Vertex AI User, Storage Object Admin, Cloud Datastore Index Admin, Secret Manager

5

Configure extension

LLM ?

Vertex AI

Gemini

Multimodal

OpenAI

Vertex AI

Other (User-provided endpoint)

Create secret

Create secret

接下來的幾項設定與集合和要嵌入的欄位有關：

LLM：Vertex AI

集合路徑：poses

預設查詢限制：3

距離測量：餘弦

輸入欄位名稱：姿勢

輸出欄位名稱：embedding

狀態欄位名稱：status

嵌入現有文件：是

更新現有嵌入：是

Cloud Functions 位置：**us-central1**

啟用事件：未勾選

× **Install this extension**
From googlecloud, in project 'Abi's'

Collection path ?

Default query limit (Optional) ?

Distance measure ?

Cosine ▼

Input field name ?

Output field name ?

Status field name ?

Embed existing documents? ?

Yes ▼

Update existing embeddings? ?

Yes ▼

完成上述設定後，請按一下「安裝擴充功能」按鈕。這項作業需要 3 到 5 分鐘。

生成嵌入：

在「poses」集中新增或更新文件時，擴充功能會透過 API 端點，使用預先訓練的模型或您選擇的模型自動生成嵌入內容。在本例中，我們已在擴充功能設定中選擇 Vertex AI。

建立索引

在應用程式中使用嵌入功能時，系統會強制在嵌入欄位上建立索引。

Firestore 會自動為基本查詢建立索引，但您也可以執行沒有索引的查詢，讓 Firestore 產生索引語法，並在應用程式端的錯誤訊息中提供產生索引的連結。以下是建立向量索引的步驟：

1. 前往 Cloud Shell 終端機
2. 執行下列指令：

```
gcloud firestore indexes composite create --collection-group="poses" --query-scope=COLLECTION
--database="(default)" --field-config vector-config='{ "dimension": "768", "flat": "
{}"}', field-path="embedding"
```

詳情請參閱[這篇文章](#)。

建立向量索引後，即可使用向量嵌入執行最鄰近搜尋。

重要注意事項：

從現在起，您不必對來源進行任何變更。只要跟著操作，就能瞭解應用程式的用途。

執行向量搜尋

讓我們看看新建立的應用程式如何處理向量搜尋。儲存嵌入項目後，您可以使用 Firestore Java SDK 的 `VectorQuery` 類別執行向量搜尋，並取得最鄰近結果：

```
CollectionReference coll = firestore.collection("poses");
VectorQuery vectorQuery = coll.findNearest(
    "embedding",
    userSearchTextEmbedding,
    /* limit */ 3,
    VectorQuery.DistanceMeasure.EUCLIDEAN,
    VectorQueryOptions.newBuilder().setDistanceResultField("vector_distance")
        .setDistanceThreshold(2.0)
        .build());
ApiFuture<VectorQuerySnapshot> future = vectorQuery.get();
VectorQuerySnapshot vectorQuerySnapshot = future.get();
List<Pose> posesList = new ArrayList<Pose>();
// Get the ID of the closest document (assuming results are sorted by distance)
String closestDocumentId = vectorQuerySnapshot.getDocuments().get(0).getId();
```

這個程式碼片段會比較使用者搜尋文字的嵌入項目與 Firestore 中文件的嵌入項目，並擷取在脈絡上最接近的項目。

7. Gemini 2.0 Flash

整合 Gemini 2.0 Flash (用於生成說明)

讓我們看看新建立的應用程式如何處理 Gemini 2.0 Flash 整合，以生成說明。

假設管理員使用者 / 瑜珈老師想在 Gemini 2.0 Flash 的協助下輸入體式詳細資料，然後執行搜尋，找出最接近的結果。因此，系統會擷取相符姿勢的詳細資料，以及支援結果的多模態物件。

```
String apiUrl = "https://generativelanguage.googleapis.com/v1beta/models/gemini-2.0-flash-exp:generateContent?key=[YOUR_API_KEY]";
Map<String, Object> requestBody = new HashMap<>();
List<Map<String, Object>> contents = new ArrayList<>();
List<Map<String, Object>> tools = new ArrayList<>();
Map<String, Object> content = new HashMap<>();
List<Map<String, Object>> parts = new ArrayList<>();
Map<String, Object> part = new HashMap<>();
part.put("text", prompt);
parts.add(part);
content.put("parts", parts);
contents.add(content);
requestBody.put("contents", contents);
/**Setting up Grounding*/
Map<String, Object> googleSearchTool = new HashMap<>();
googleSearchTool.put("googleSearch", new HashMap<>());
tools.add(googleSearchTool);
requestBody.put("tools", tools);

HttpHeaders headers = new HttpHeaders();
headers.setContentType(MediaType.APPLICATION_JSON);
HttpEntity<Map<String, Object>> requestEntity = new HttpEntity<>(requestBody, headers);
ResponseEntity<String> response = restTemplate.exchange(apiUrl, HttpMethod.POST,
requestEntity, String.class);
System.out.println("Generated response: " + response);
String responseBody = response.getBody();
JSONObject jsonObject = new JSONObject(responseBody);
JSONArray candidates = jsonObject.getJSONArray("candidates");
JSONObject candidate = candidates.getJSONObject(0);
JSONObject contentResponse = candidate.getJSONObject("content");
JSONArray partsResponse = contentResponse.getJSONArray("parts");
JSONObject partResponse = partsResponse.getJSONObject(0);
String generatedText = partResponse.getString("text");
System.out.println("Generated Text: " + generatedText);
```

a. 模仿圖像和音訊生成

Gemini 2.0 Flash Experimental 能夠生成多模態結果，但我尚未註冊搶先體驗，因此分別使用 Imagen 和 TTS API 模擬圖像和音訊輸出。試想一下，只要對 Gemini 2.0 Flash 進行一次 API 呼叫，就能生成所有這些內容，是不是很棒！


```

try (PredictionServiceClient predictionServiceClient =
    PredictionServiceClient.create(predictionServiceSettings)) {

    final EndpointName endpointName =
        EndpointName.ofProjectLocationPublisherModelName(
            projectId, location, "google", "imagen-3.0-generate-001");

    Map<String, Object> instancesMap = new HashMap<>();
    instancesMap.put("prompt", prompt);
    Value instances = mapToValue(instancesMap);

    Map<String, Object> paramsMap = new HashMap<>();
    paramsMap.put("sampleCount", 1);
    paramsMap.put("aspectRatio", "1:1");
    paramsMap.put("safetyFilterLevel", "block_few");
    paramsMap.put("personGeneration", "allow_adult");
    Value parameters = mapToValue(paramsMap);

    PredictResponse predictResponse =
        predictionServiceClient.predict(
            endpointName, Collections.singletonList(instances), parameters);

    for (Value prediction : predictResponse.getPredictionsList()) {
        Map<String, Value> fieldsMap = prediction.getStructValue().getFieldsMap();
        if (fieldsMap.containsKey("bytesBase64Encoded")) {
            bytesBase64Encoded = fieldsMap.get("bytesBase64Encoded").getStringValue();
        }
    }
    return bytesBase64Encoded;
}

```

```

try {
    // Create a Text-to-Speech client
    try (TextToSpeechClient textToSpeechClient = TextToSpeechClient.create()) {
        // Set the text input to be synthesized
        SynthesisInput input =
SynthesisInput.newBuilder().setText(postureString).build();

        // Build the voice request, select the language code ("en-US") and the ssml
        // voice gender
        // ("neutral")
        VoiceSelectionParams voice =
            VoiceSelectionParams.newBuilder()
                .setLanguageCode("en-US")
                .setSsmlGender(SsmlVoiceGender.NEUTRAL)
                .build();

        // Select the type of audio file you want returned
        AudioConfig audioConfig =
            AudioConfig.newBuilder().setAudioEncoding(AudioEncoding.MP3).build();

        // Perform the text-to-speech request on the text input with the selected
voice
        // parameters and audio file type
        SynthesizeSpeechResponse response =
            textToSpeechClient.synthesizeSpeech(input, voice, audioConfig);

        // Get the audio contents from the response
        ByteString audioContents = response.getAudioContent();

        // Convert to Base64 string
        String base64Audio =
Base64.getEncoder().encodeToString(audioContents.toByteArray());

        // Add the Base64 encoded audio to the Pose object
        return base64Audio;
    }

    } catch (Exception e) {
        e.printStackTrace(); // Handle exceptions appropriately. For a real app, log and
provide user feedback.
        return "Error in Audio Generation";
    }
}

```

b. 以 Google 搜尋建立基準：

如果您在步驟 6 中檢查 Gemini 叫用程式碼，會發現下列程式碼片段可啟用 Google 搜尋，為 LLM 回覆建立基準：

```
/**Setting up Grounding*/  
Map<String, Object> googleSearchTool = new HashMap<>();  
googleSearchTool.put("googleSearch", new HashMap<>());  
tools.add(googleSearchTool);  
requestBody.put("tools", tools);
```

這是為了確保我們：

- 根據實際搜尋結果提供模型資訊
 - 擷取搜尋結果中提及的相關網址
-

步驟 8 · 約 0 分鐘

8. 執行應用程式

現在，我們來看看新建立的 Java Spring Boot 應用程式有哪些功能，以及簡單的 Thymeleaf 網頁介面：

1. Firestore CRUD 作業 (建立、讀取、更新、刪除)
2. 關鍵字搜尋
3. 以生成式 AI 為基礎建立情境
4. 內容比對搜尋 (向量搜尋)
5. 與搜尋內容相關的多模態輸出內容
6. 執行自己的查詢 (結構化查詢格式的查詢)

範例：{"structuredQuery":{"select":{"fields":[{"fieldPath":"name"}]}, "from":[{"collectionId":"fitness_poses"]}}

目前討論的所有功能都屬於您剛從存放區建立的應用程式：<https://github.com/AbiramiSukumaran/firestore-poserecommender>

如要建構、執行及部署，請在 Cloud Shell 終端機執行下列指令：

```
mvn package  
  
mvn spring-boot:run
```

您應該會看到結果，並能試用應用程式功能。如要查看輸出內容的示範，請觀看下方影片：

[運用 Firestore、Vector Search 和 Gemini 2.0 Flash 打造體式推薦工具](#)

選用步驟：

如要在 Cloud Run 上部署應用程式 (假設您已使用 Dockerfile 啟動全新應用程式，並視需要複製檔案)，請在專案目錄中，從 Cloud Shell 終端機執行下列指令：

```
gcloud run deploy --source .
```

提供應用程式名稱、地區代碼 (選擇 **us-central1** 的代碼)，並在系統提示時選擇未經驗證的叫用「Y」。部署成功後，終端機中應該會顯示應用程式端點。

步驟 9 · 約 0 分鐘

9. 清除所用資源

如要避免系統向您的 Google Cloud 帳戶收取本文章所用資源的費用，請按照下列步驟操作：

1. 在 Google Cloud 控制台中前往「管理資源」頁面。
 2. 在專案清單中選取要刪除的專案，然後點按「刪除」。
 3. 在對話方塊中輸入專案 ID，然後按一下「Shut down」(關機) 即可刪除專案。
-

10. 恭喜

恭喜！您已成功運用 Firestore 建立強大且智慧的瑜珈姿勢管理應用程式。我們結合 Firestore 的強大功能、Vector Search 擴充功能和 Gemini 2.0 Flash 的功能 (包括模擬圖片和音訊生成)，打造出真正引人入勝且資訊豐富的瑜珈應用程式，可實作 CRUD 作業、執行關鍵字搜尋、情境向量搜尋，以及生成多媒體內容。

這種做法不限於 Yoga 應用程式。隨著 Gemini 等 AI 模型持續發展，我們將能打造更身歷其境的個人化使用者體驗。請記得隨時掌握 Google Cloud 和 Firebase 的最新發展和說明文件，充分發揮這些技術的潛力。

如果想擴充這個應用程式，我會嘗試使用 [Gemini 2.0 Flash](#) 執行下列兩項操作：

1. 建立即時影像和音訊串流，運用 [Multimodal Live API](#) 解決用例。
2. 啟用[思考模式](#)，讓 AI 根據即時資料生成回覆背後的想法，提供更貼近現實的體驗。

歡迎試試看，並傳送提取要求 :>D!!!
